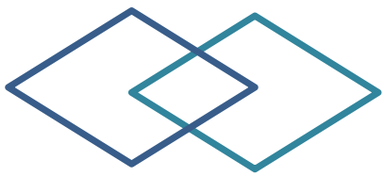


第11 章

从模块到项目

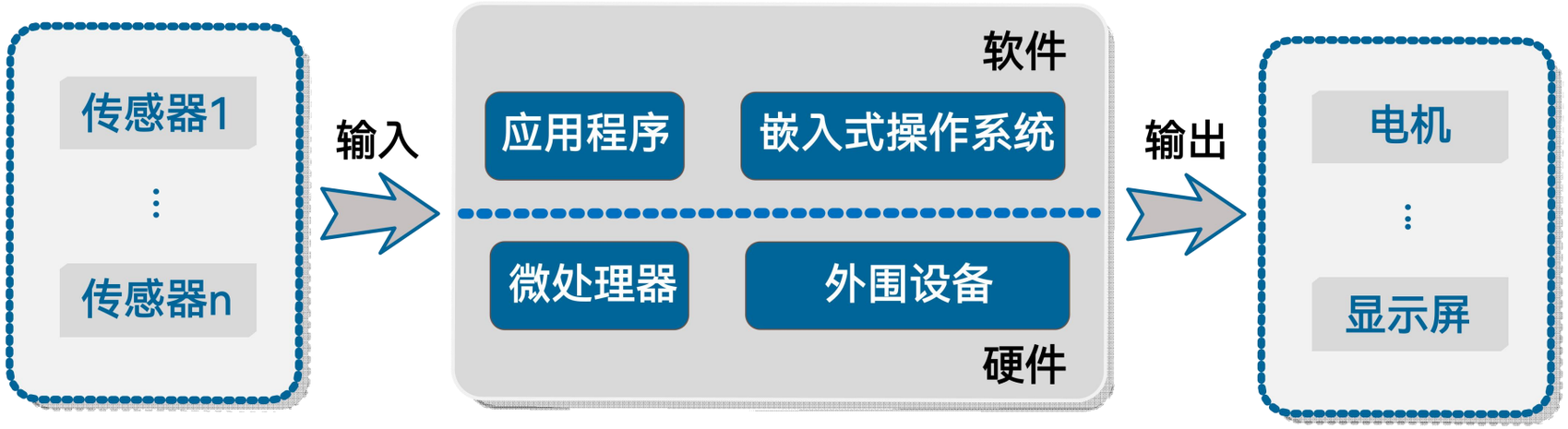


本章知识与能力要求

- 理解项目与模块的关系；
- 理解从模块到项目的设计思想；
- 掌握由模块构成项目的方法。

第11章 从模块到项目

- 项目是由两个或两个以上的功能模块组成的系统，是功能模块的有机融合。
- 工作原理：处理输入事件，经微处理器加工处理，产生结果并输出。
- 其输入事件往往来自传感器、按键或触屏等外围设备，处理后，输出信号用于驱动电机等执行机构或显示。



第11章 从模块到项目

目 录


11.1 从模块到项目


11.2 项目实践


11.3 面向对象的编程思想

11.1 如何从模块到项目



11.1.1 项目需求分析



11.1.2 系统总体架构设计



11.1.3 系统软硬件设计

11.1.1 项目需求分析

对综合项目进行需求分析，明晰功能，明确一个系统/项目的输入输出，弄清楚项目/系统的应用范围或详细的规格要求。

- 项目/系统需要达到的各项性能指标有哪些；
- 系统功能目前有哪些可行的解决方案；
- 系统的功耗、体积有何要求；
- 微控制器如何选型，传感器采用数字的还是模拟的？
- 输出显示设备是采用OLED还是上位机(PC)显示？
- 系统运行时突发故障的处理。

这一阶段思考得越详细、考虑得越深入，对后期的开发就越有利。

11.1.2 系统总体架构设计

对系统进行功能模块划分、软件和硬件划分。

软件和硬件的划分

- 硬件实现成本高，但执行效率高；
- 软件实现开发难度大，但成本低，

系统功能模块的划分

采用自顶向下、逐步细分为功能相对独立的模块化设计，会使得程序框架更为清晰。

11.1.2 系统总体架构设计

一个嵌入式系统通常包括两大类模块

硬件驱动模块

硬件模块如电源模块、通信模块、存储模块、显示模块等

一种硬件对应一个硬件驱动模块

实现硬件的初始化操作，通常使用相应硬件名称命名，如LED硬件驱动模块定义为led.c



应用功能模块

多以具体功能进行划分，功能模块之间应满足低偶合、高内聚的要求

高内聚：一个C文件中函数，只有相互之间的调用，而不存在调用其它文件中函数的情况

低耦合：模块与模块之间应相对独立，模块与模块之间的接口应尽量少而简单。

11.1.2 系统总体架构设计

嵌入式系统应遵循的编程原则

5条

1．每一个功能模块都由源文件.c和头文件.h组成

2．每一个.c源文件都对应一个同名的.h头文件

3．使用头文件共享函数、变量

4．使用头文件共享宏及类型定义

5．防止头文件被重复包含

11.1.2 系统总体架构设计

嵌入式系统应遵循的编程原则

1. 每一个功能模块都由源文件.c和头文件.h组成

- .c源文件是每个功能模块具体功能的实现。
- .h头文件是对该模块接口的声明，对外提供接口供外部程序调用，

2．每一个.c源文件都对应一个同名的.h头文件

把相对独立的功能放在.c源文件中用代码实现，把需要对外使用的函数或变量声明放在对应的.h头文件中供其它模块使用。

3．使用头文件共享函数、变量

将某模块提供给其它模块调用的函数、变量在对应模块的.h头文件中用extern关键字声明，而模块内的函数和全局变量应在.c文件开头部分用static关键字声明。

11.1.2 系统总体架构设计

嵌入式系统应遵循的编程原则

4．使用头文件共享宏及类型定义

- 针对复杂的系统，由于模块较多，涉及到的文件多，模块间全局变量也较多，可以将全局变量定义在一个.h头文件内

例如：将寄存器的位操作及常用的数据类型用typedef定义在头文件中，方便以后调用以及移植修改。

```
#ifndef __SYS_H
#define __SYS_H

#include "stm32f10x.h"
#include "stdio.h"
            //位操作宏定义
#define  BIT0                    (0x0001)
#define  BIT1                    (0x0002)
… …
#define  BIT30                   (0x40000000)
#define  BIT31                   (0x80000000)
```

11.1.2 系统总体架构设计

嵌入式系统应遵循的编程原则

4．使用头文件共享宏及类型定义

```
 //常用的数据类型定义，采用宏定义方式，便于移植和更改
#define  u8      uint8_t
#define  u16     uint16_t
#define  u32     uint32_t
#define  s8      int8_t
#define  s16     int16_t
#define  s32     int32_t
#define  bool    _Bool
#define  false   0
#define  true    !false
```

```
    //结构体定义
typedef struct
{    unsigned short b0:1;
     unsigned short b1:1;
     … …
     unsigned short b15:1;
}REG16_TypeDef;
     /*位带操作，实现51类似的GPIO控制功能*/
     //IO口宏定义操作
#define  PAout ((volatile REG16_TypeDef
*)GPIOA_ODR_Addr)
     //输出
#define  Pain   ((volatile REG16_TypeDef
*)GPIOA_IDR_Addr)
     //输入
                 … …
#define  PFout   ((volatile REG16_TypeDef
*)GPIOF_ODR_Addr)
     //输出
#define  Pfin   ((volatile REG16_TypeDef
*)GPIOF_IDR_Addr)
     /输入
```

11.1.2 系统总体架构设计

嵌入式系统应遵循的编程原则

5．防止头文件被重复包含

- 每个模块都对应有.h头文件，为了避免头文件被重复包含，导致编译错误，通常使用条件编译命令

  #ifndef … …#endif和#define宏定义实现。

模块化设计中头文件的一般格式

```
#ifndef __XXX_H
#define __XXX_H
#include "stm32f10x.h"
#ifdef IN_XXX
   #define XXX_EXT
#else
   #define XXX_EXT          extern
#endif
#define  Speaker            PBout->b9
#pragma pack(1)
//伪指令# pragma pack(n)的作用是自定义字节对齐方式，n为字节个数
```

11.1.2 系统总体架构设计

嵌入式系统应遵循的编程原则

5．防止头文件被重复包含

```
    //结构体定义
typedef struct
{
volatile u32  Weight_ADC;

volatile u8   Device_BH[10];

volatile u16  Alarm_Time;
volatile u8   Lock_State;
}Device_Type;
#pragma pack( )
//取消自定义字节对齐方式
```

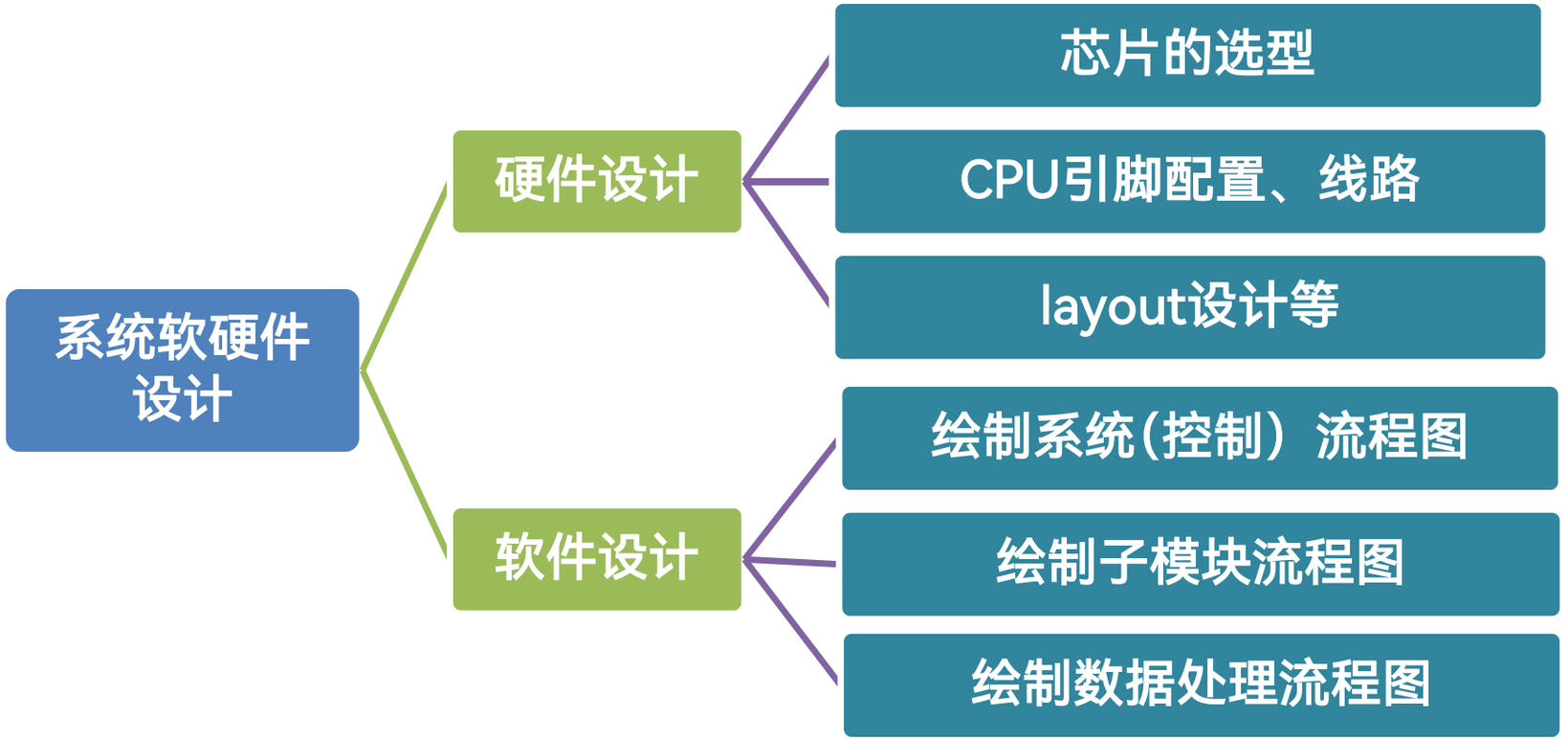
```
//声明外部变量
MAIN_EXT  volatile u8   Device_State;

MAIN_EXT  volatile u16  Weight;

MAIN_EXT  volatile u32
Weight_Buf[20];
//声明外部函数
void Sys_Init(void);
void On_MeasureProcess(void);
void On_SpeakAlarm(void);
#endif
```

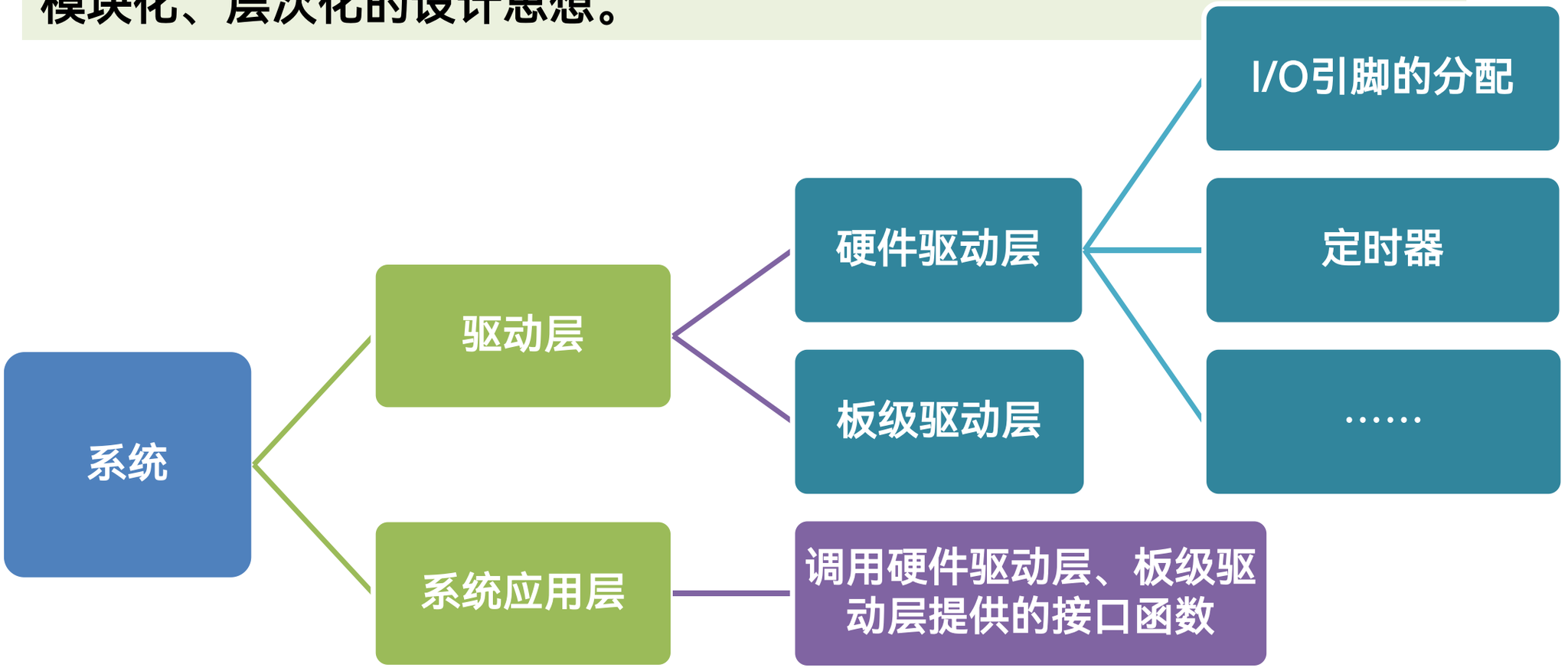
11.1.3 系统软硬件设计

系统总体架构设计确定后，接下来就需要进行系统的详细设计，即具体的硬件设计和软件设计。



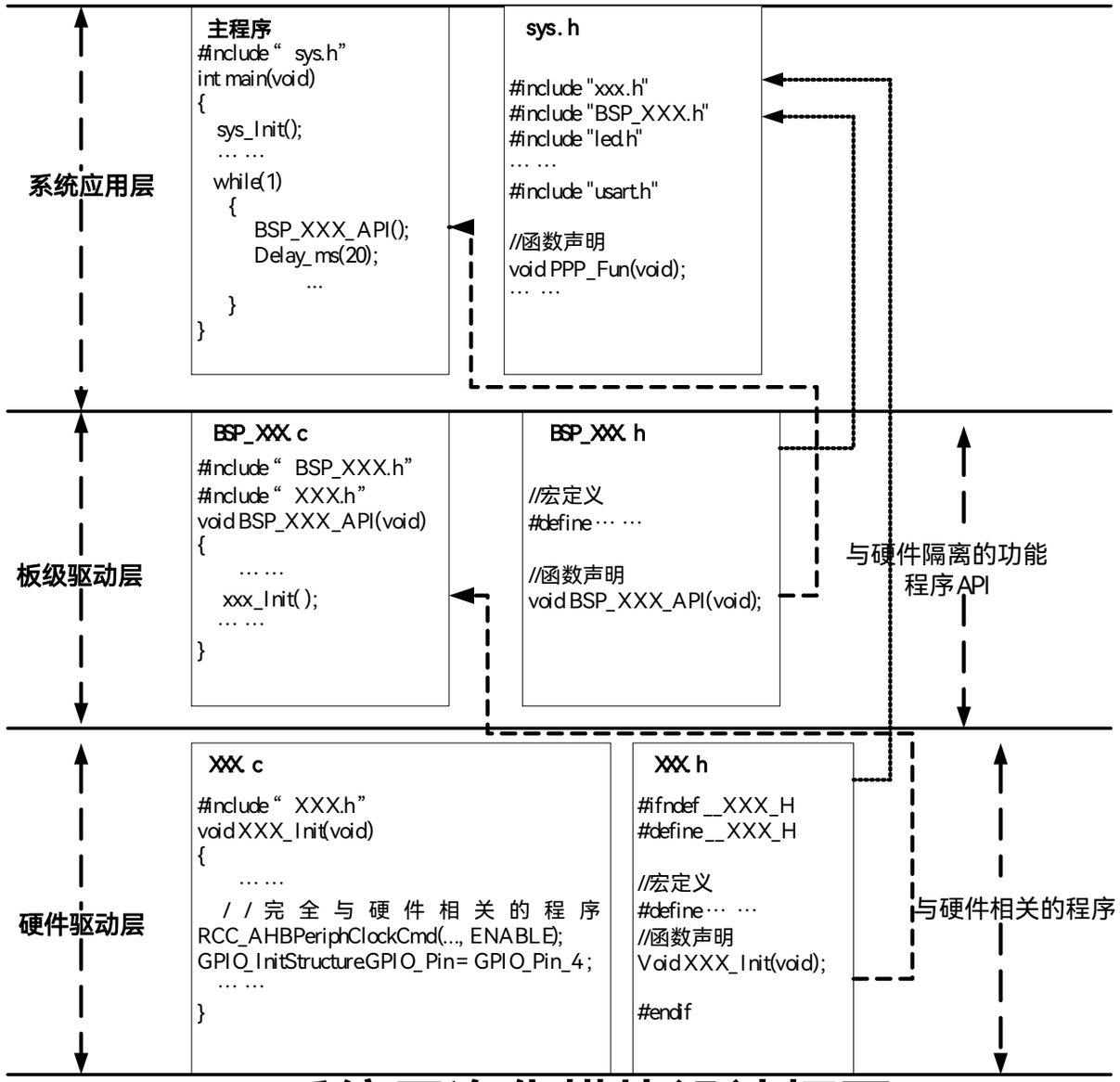
11.1.3 系统软硬件设计

可扩充性和可移植性反映在系统架构设计中，就是系统必须采用模块化、层次化的设计思想。



11.1.3系统软硬件设计

- 将与硬件相关的模块和与硬件无关的模块明确区分开来；
- 各模块之间通过API函数进行沟通；
- 实现系统层次化、功能模块化；
- 有利于调试阶段的系统整合以及后续的系统维护和升级改造。



系统层次化模块设计框图

11.2 项目实践

项目实践



01 11.2.1 项目需求分析

02 11.2.2 项目总体架构设计

03 11.2.3 系统硬件设计

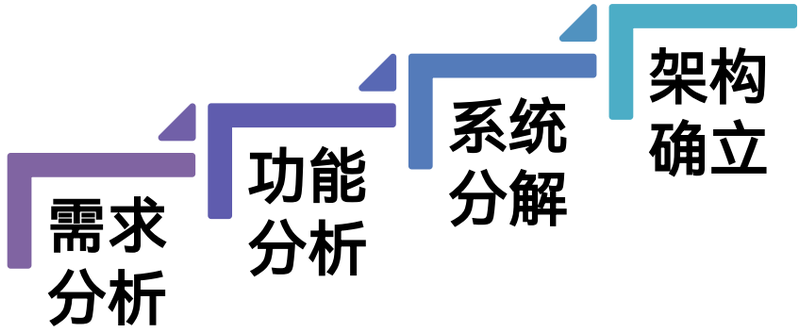
04 11.2.4 系统软件设计

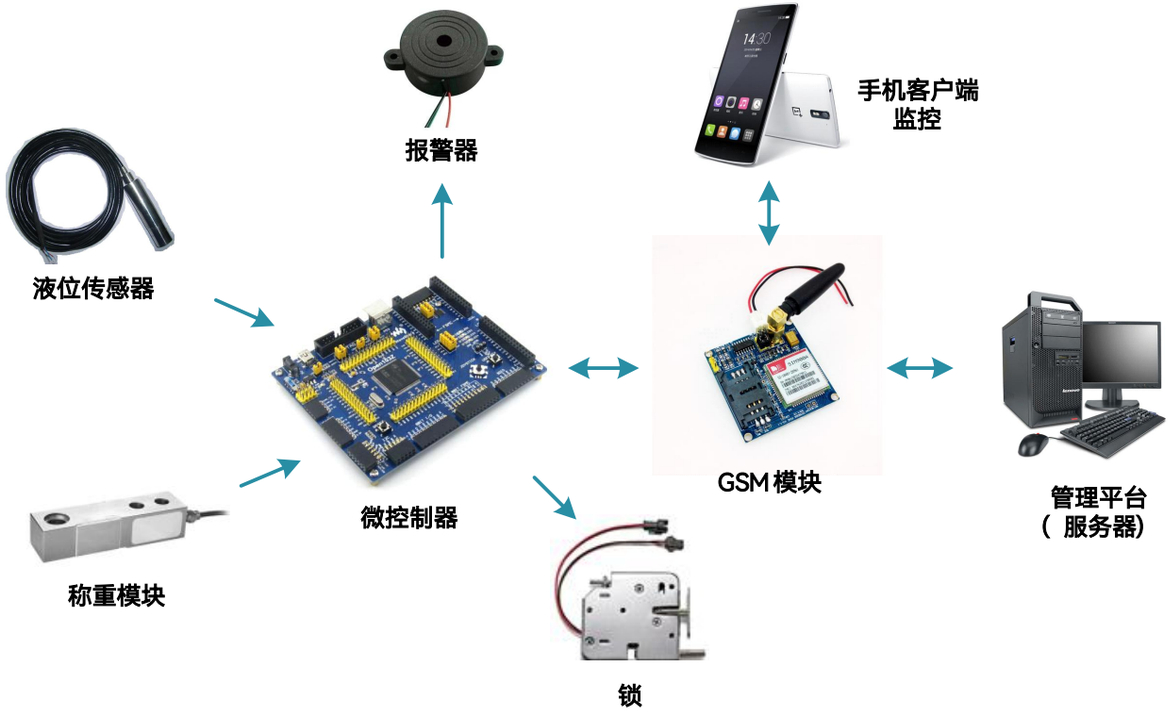
11.2.1 项目案例背景分析



11.2.2 项目总体架构设计

系统总体架构设计是把系统功能需求分析的结果转换为系统体系结构，形成系统模块结构图。

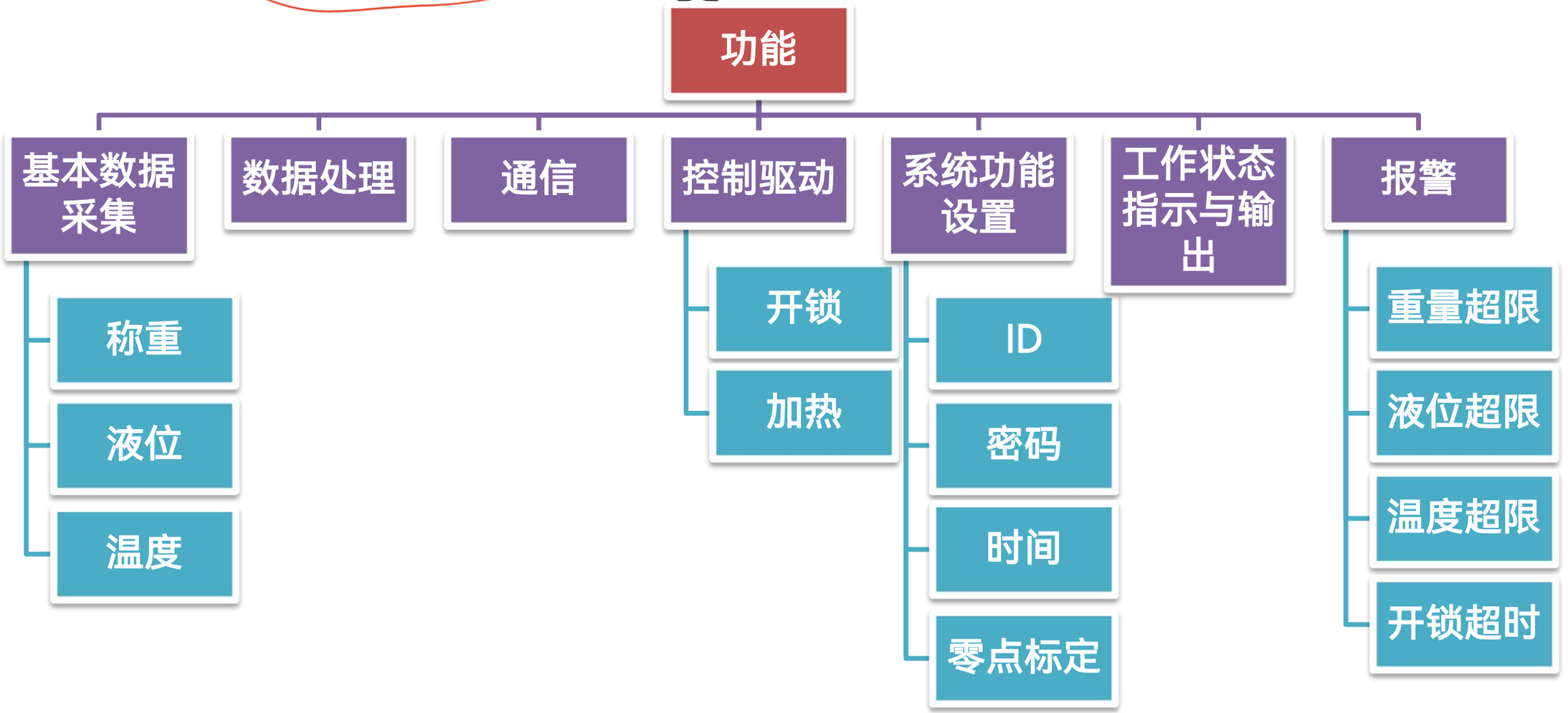




系统功能示意图

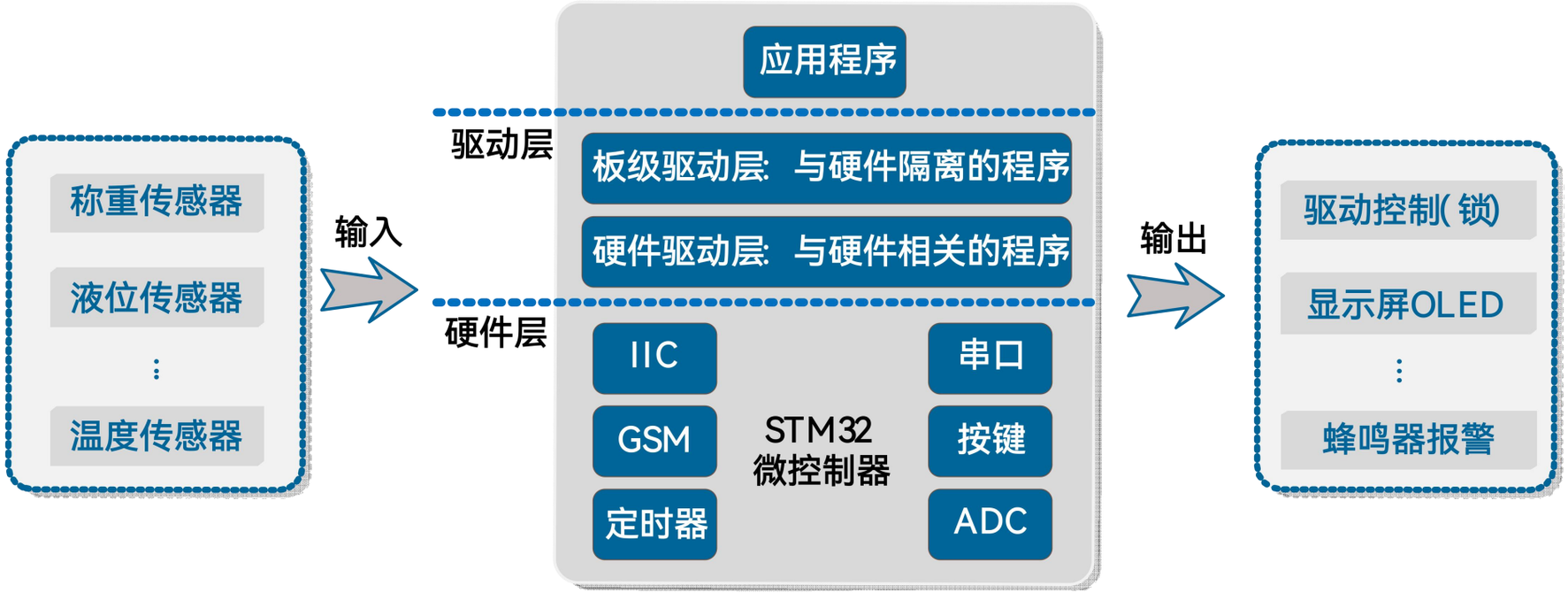
11.2.2 项目总体架构设计

采用ST公司的STM32F103系列微处理器作为主控芯片，实现餐厨垃圾智能监测系统的前端设备



11.2.2 项目总体架构设计

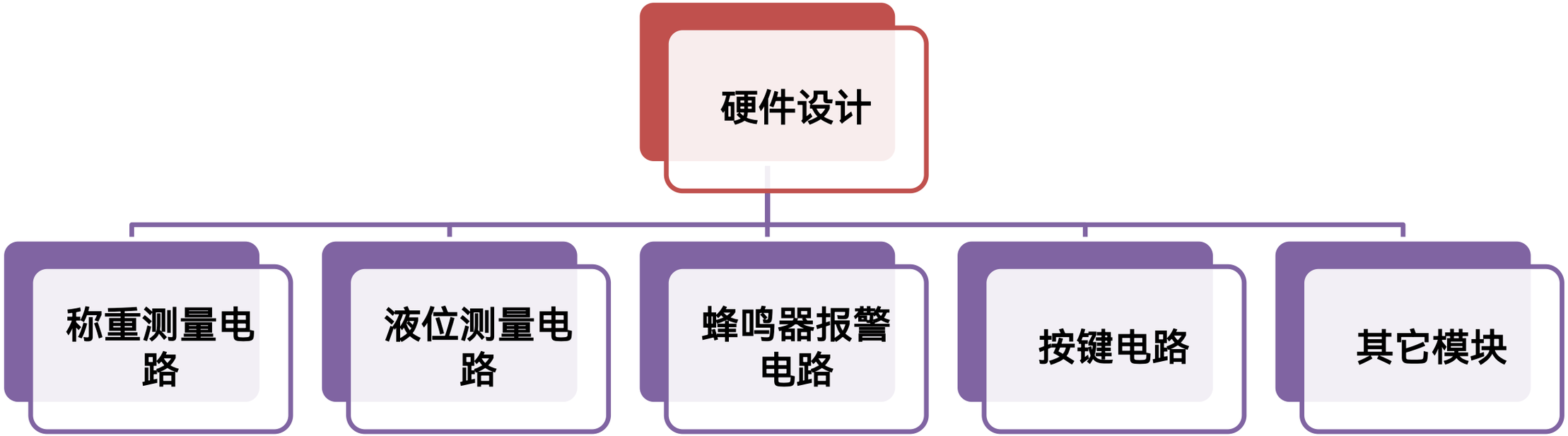
嵌入式系统开发有裸机开发(单片机的开发模式）和基于嵌入式操作系统(RTOS）开发两种，本系统模块少，流程简单，采用**基于裸机的开发**更为适合。



餐厨垃圾智能监测系统设计框图

11.2.3 系统硬件设计

主要涉及各硬件模块的I/O引脚配置、微控制器最小系统设计(晶振、调试和复位电路等）以及电源电路的设计等。

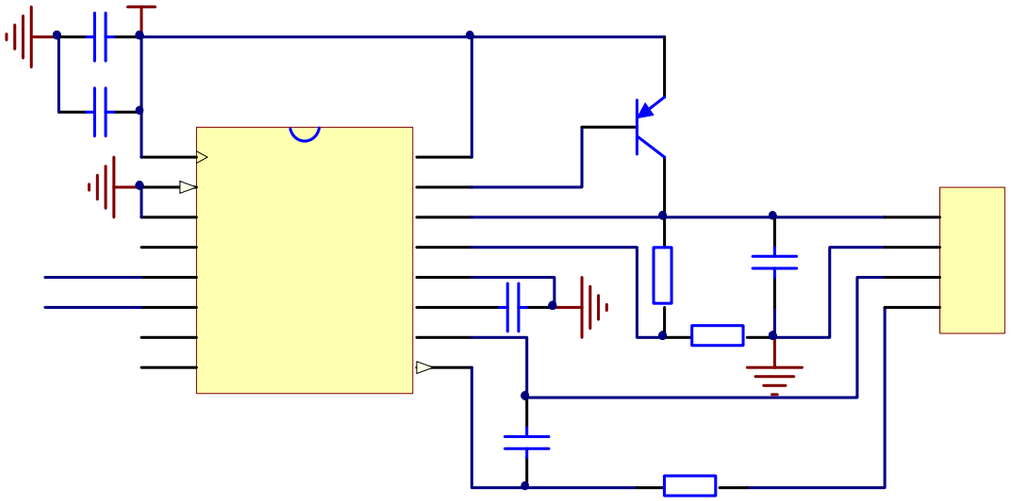


11.2.3 系统硬件设计

称重测量电路

利用HX711模块采集废弃油脂收集桶的压力信号，用于废弃油脂的称重。

HX711是一款专为高精度电子秤设计的24位AD转换器芯片，内部集成了稳压电源、时钟振荡器等AD转换所必需的电路，具有集成度高、响应速度快、抗干扰能力强等优点，提高了性能和可靠性。

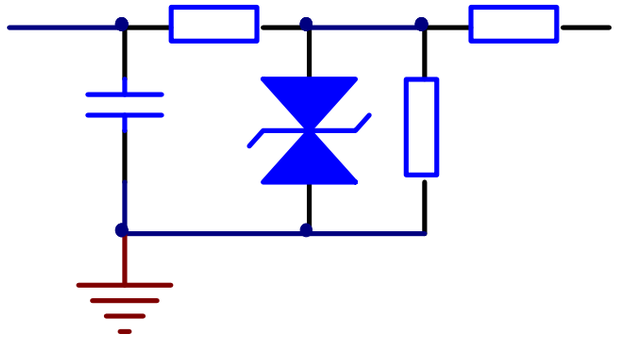


称重传感器信号采用差分输入模式从通道A输入，与引脚INNA(通道A负输入端）和INPA(通道A正输入端）相连接。

11.2.3 系统硬件设计

液位测量电路

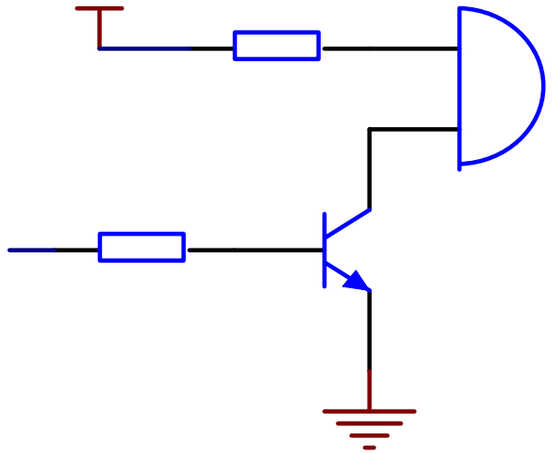
- 液位检测采用投入式液位变送器，输出4～20mA标准模拟电流信号；
- AD转换过程则由微控制器内部集成的AD转换器实现；



电路中的TVS管SMBJ5.0CA用于消除瞬时脉冲对电路的影响，经过调理后的液位传感器信号连接到微控制器的I/O引脚。

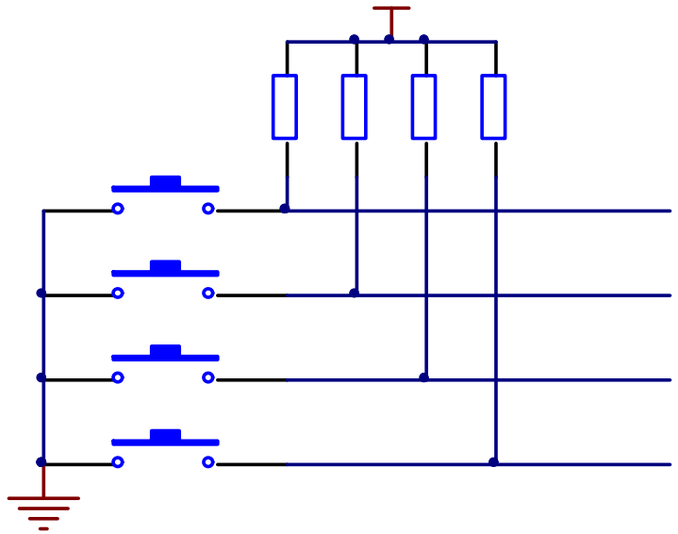
11.2.3 系统硬件设计

蜂鸣器报警电路



需要报警时微控制器经I/O引脚送出低电平，三极管Q1导通，蜂鸣器报警

按键电路



按键模块用于设置系统参数，比如设置当前日期、压力传感器的参数标定等

11.2.4 系统软件设计

嵌入式系统的开发需要考虑两个特性，一个是可扩充性，一个是可移植性。

系统必须采用**模块化、层次化**的设计思想：

- 将与硬件相关的部分和与硬件无关的模块分开；
- 各模块之间的信息交互通过API来实现。

驱动层

完全与硬件相关的硬件驱动层

- 与所用硬件平台密切相关，主要涉及微控制器I/O引脚的分配、微控制器内部定时器、串口和ADC等功能模块的使用，实现接口的驱动功能，可移植性和可重用性较低；

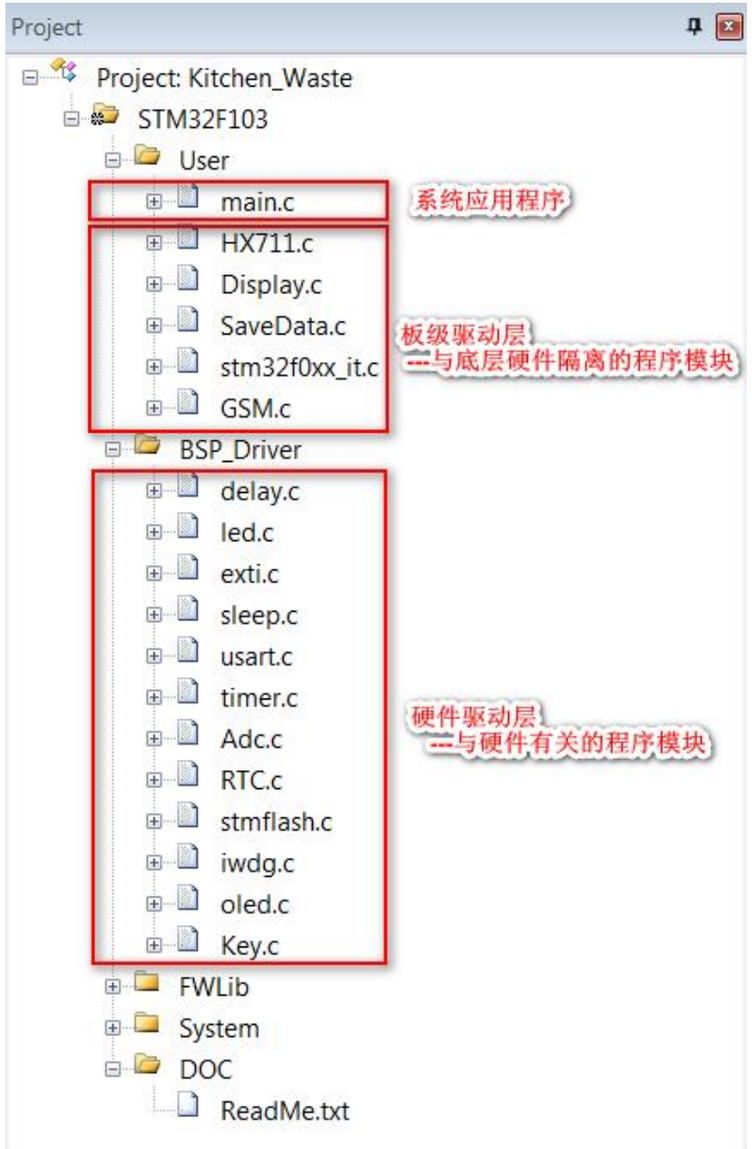
与底层硬件隔离的板级驱动层

- 通过调用底层硬件驱动提供的API函数实现硬件隔离，用于隐藏具体的硬件特性，并提供API供系统应用层或其它程序模块调用。

11.2.4 系统软件设计

硬件驱动层

- 硬件驱动层的主要是各硬件驱动模块的划分和设计，涉及到.c和.h的模块化划分思想。
- 系统应用程序层的设计主要是全局变量的定义、系统功能函数实现等，涉及到流程图的绘制、C语言函数模块规范化编写规范等。
- 每个功能模块对应一个相应的头文件。



餐厨垃圾监测系统工程文件架构图

11.2.4 系统软件设计

key.c

```
#define  IN_KEY
#include "key.h"
#include "delay.h"
void  KEY_Init(void)  //按键初始化函数
{   GPIO_InitTypeDef   GPIO_InitStructure;
    RCC_AHBPeriphClockCmd(RCC_AHBPeriph_GPIOB,
ENABLE);
     //按键
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_11 |
GPIO_Pin_12 | GPIO_Pin_13 | GPIO_Pin_14;
   GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
   GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
   GPIO_Init(GPIOB, &GPIO_InitStructure);
}
//按键处理函数
u8 KEY_Scan(void)
{    static u8 key_up=1;//按键按松开标志
```

```
 if(key_up==1
&&(KEY_SET==0||KEY_ADD==0||KEY_DEC==0||K
EY_OK==0))
    {   delay_ms(50);//去抖动
         key_up = 0;
        if(KEY_SET==0)   return 1;
        if(KEY_ADD==0)    return 2;
        if(KEY_DEC==0)    return 3;
       if(KEY_OK==0)    return 4;
    }
    else
if(KEY_SET==1&&KEY_ADD==1&&KEY_DEC==1
&&KEY_OK==1)
   {   if(key_up==0)
       {   key_up = 1;      }
     }
  return 0; //无按键按下
}
```

11.2.4 系统软件设计

key.h

```
#ifndef __KEY_H
#define __KEY_H

#include "sys.h"

#ifdef  IN_KEY
    #define  KEY_EXT
#else
    #define  KEY_EXT          extern
#endif
```

```
 #define  KEY_SET          PBin->b11
#define  KEY_ADD  PBin->b12
#define  KEY_DEC          PBin->b13
#define  KEY_OK    PBin->b14

void  KEY_Init(void);  //按键I/O引脚初始化
u8   KEY_Scan(void);  //按键扫描函数，用于判断是哪个按键操作
#endif
```

11.2.4 系统软件设计

板级驱动层划分为：称重测量模块、蓝牙或GSM通信控制模块、功能菜单显示模块等

```
u32  HX711_ReadCount(void) //读取HX711的AD数值
{   u32 i,Count;
    HX711_SCK = 0;
      //使能AD
    Count = 0;
   while(HX711_DT==1);
       //AD转换未结束则等待，否则开始读取
      for(i=0; i<24; i++)
      {    HX711_SCK = 1;   //PD_SCK置高(发送脉冲)
           Count <<= 1;
             //下降沿来时变量Count左移一位，右侧补零
           HX711_SCK = 0;
               //PD_SCK 置低
            if(HX711_DT==1)              Count++;

       }
       HX711_SCK = 1;
      Count = Count^0x800000;
       //第25个脉冲下降沿来时，转换数据
      HX711_SCK = 0;
```

HX711.h

```
#ifndef __HX711_H
#define __HX711_H
#include "sys.h"

#ifdef  IN_HX711
    #define  HX711_EXT
#else
    #define  HX711_EXT  extern
#endif

u32  HX711_ReadCount(void);

    //读取HX711的AD数值
#endif
```

11.2.4 系统软件设计

系统应用程序

功能

- 通过调用硬件驱动层提供的接口函数实现相应的功能;
- 模块的划分以“信息处理”为原则。以“数据流导向”方式体现在代码中

工具

- 程序流程图
- 绘制各层级的流程图
- 图形化的表示方式，直观清晰

编写规范

- 函数头添加相应的注释，比如函数名称、函数功能、函数入口参数、返回值等的说明
- 遵循一定的程序编写规范

11.3 面向对象的编程思想

- 对问题进行自然分割，问题抽象成若干个对象
- 同类型的对象具有相同的属性和方法，在对象中封装属性和方法，所有的业务操作变为对象的行为和对象之间的消息传递



- 将接口抽象成类，程序通过各个类的实例实现通信和协作，形成通用的接口，实现代码的可拓展性，满足系统对模块的低耦合、高内聚要求。
- 面向对象编程涉及封装、继承和多态等概念，在嵌入式系统中多采用结构体、指针和枚举等数据类型来实现。

本章小结

11.1 如何从模块到项目

项目需求分析

总体架构设计

软硬件设计

11.2 项目实践

背景分析

总体架构设计

硬件设计

软件设计

11.2 面向对象编程思想



The End!

